EXERCÍCIO B1 — ARCHITECTURE LAB

Seis desafios de decisão arquitetural — 6,0 pontos

Respostas da equipe

Gustavo Gutierres Champam — RA 223645

Gustavo Oliveira Camargo — RA 236024

Respondemos aos seis desafios com base nas microsoluções e evidências do repositório. Escolhemos manter o Core modular e separar as integrações conforme a necessidade de cada fluxo. Indicamos nas respostas o que foi testado e o que ainda precisa ser validado na AF.

Repositório da equipe

1. Contexto e propósito

Na AC1, a equipe construiu um MVP de Gamificação para Educação Continuada. No B1, esse mesmo produto passa a receber crescimento de usuários, novas capacidades de IA, eventos, IoT e exigências de operação. A equipe assume o papel de Tech Lead: antes de escolher tecnologia ou alterar o código, deverá justificar a decisão arquitetural a partir dos requisitos e das evidências do sistema.

O objetivo não é produzir seis arquiteturas diferentes nem acumular tecnologias. O objetivo é demonstrar raciocínio arquitetural consistente e evolutivo. Cada desafio vale 1,0 ponto e deverá deixar uma decisão rastreável no repositório.

2. Padrão obrigatório de decisão — usado nos 6 desafios

1. RF	2. RNF	3. ASR	4. RPC	5. Alternativas	6. ADR	7. C4	8. Evidência

RF = requisito funcional; RNF = requisito não funcional, classificado quando pertinente por FURPS+; ASR = requisito arquiteturalmente significativo; RPC = requisito/premissa/constraint que restringe a solução (tecnologia, legislação, equipe, legado, protocolo, infraestrutura ou integração).

Pergunta central do Tech Lead

Qual decisão arquitetural é necessária para atender aos RFs, proteger os RNFs que realmente dirigem a arquitetura, respeitar as RPCs e aceitar conscientemente os trade-offs?

3. Regras do Architecture Lab

- Toda tecnologia escolhida precisa responder a um problema do case. “É moderno”, “é mais escalável” ou “o mercado usa” não são justificativas suficientes.
- Nem todo RNF é um ASR. A equipe deverá selecionar apenas os RNFs capazes de alterar estrutura, comunicação, dados, implantação ou operação.
- RPC não é solução: é uma restrição ou premissa que limita o espaço de decisão. Ex.: Core deve permanecer em Java/Spring Boot; IA será implementada em Python.
- O ADR registra a decisão, alternativas e consequências. O C4 representa a arquitetura resultante; ele não substitui o raciocínio da decisão.
- A microsolução é evidência técnica. Ela deve testar uma hipótese arquitetural, e não apenas demonstrar que uma ferramenta funciona.

4. Case-base

A plataforma possui ranking, cursos, moedas, badges e recompensas. O Core existente utiliza Java/Spring Boot e banco relacional. O produto deverá evoluir para múltiplas instituições, recomendação inteligente, assistente educacional com RAG/LLM, notificações, eventos de aprendizagem e integração com dispositivos IoT, mantendo confiabilidade, testabilidade, observabilidade e evolução contínua.

EXERCÍCIO B1 — COMO ENTREGAR

Um único registro arquitetural vivo no Git

5. O que deve existir ao final de cada desafio

Registro	Conteúdo mínimo	Pergunta de validação
RF/RNF	RFs afetados + RNFs relevantes/FURPS+	Qual necessidade ou qualidade está em jogo?
ASR	Máx. 3 ASRs prioritários no desafio	O que realmente pode mudar a arquitetura?
RPC	Restrições/premissas aplicáveis	O que não pode ser ignorado na decisão?
Alternativas	Mín. 2 alternativas + trade-offs	Que opções reais foram comparadas?
ADR	Contexto, decisão e consequências	Por que esta alternativa foi escolhida?
C4	Atualizar somente o nível afetado	O desenho mostra a decisão tomada?
Evidência	Mini-lab, teste, métrica, falha simulada etc.	Que evidência sustenta a decisão?

6. Registro no repositório

Manter /docs/arquitetura/README.md como registro cumulativo. A matriz de ASRs deve evoluir ao longo dos seis desafios; os ADRs podem permanecer no README (ADR-001, ADR-002...) ou ser separados em /docs/adr/. O C4 Container é a visão mínima obrigatória; C4 Component será utilizado quando a decisão envolver a organização interna do Core.

7. Avaliação — 6,0 pontos

Cada desafio vale 1,0 ponto e será avaliado pelo mesmo padrão. A nota não depende de adotar a tecnologia sugerida no cenário, mas da coerência da decisão e da evidência apresentada.

Critério por desafio	Valor
RF/RNF + seleção de ASR	**0,25**
RPC + alternativas/trade-offs	**0,20**
ADR e justificativa da decisão	**0,20**
C4 coerente com a decisão	**0,15**
Microsolução/evidência + Git	**0,20**

8. Entrega final do B1

- Link do repositório e identificação da equipe.
- README arquitetural atualizado com as seis decisões e matriz cumulativa de ASRs.
- ADRs essenciais e C4 final coerente com as decisões.
- Links/pastas das microsoluções e evidências.
- Síntese final: quais decisões serão levadas para a PoC da Avaliação Final (AF).

Regra de ouro: uma decisão forte consegue ser lida de trás para frente — o C4 é explicado pelo ADR; o ADR é sustentado pelos ASRs e RPCs; os ASRs derivam de RF/RNF; e a evidência demonstra que o raciocínio é plausível.

DESAFIO 1 — DRIVERS E EVOLUÇÃO DO CORE

Decidir a arquitetura macro antes de decompor o sistema

Cenário / problema

O MVP cresceu de 1.000 para um cenário de até 100.000 usuários. O ranking cresce mais que outras funcionalidades, enquanto cursos, conquistas e recompensas continuam evoluindo. A equipe precisa decidir se mantém o Monólito em Camadas, evolui para Monólito Modular ou separa capacidades em serviços.

1. RF — o que o sistema precisa fazer

RF01 Ranking; RF02 Conquistas; evolução das funcionalidades existentes.

Resposta da equipe

Mantemos RF01 Ranking, RF02 Conquistas e as funcionalidades de cursos, moedas e recompensas. A evolução deve preservar a regra de pontuação e permitir mudanças no ranking sem espalhar alterações pelo sistema.

2. RNF — qualidades a analisar

Manutenibilidade, desempenho, escalabilidade, testabilidade e complexidade operacional.

Resposta da equipe

Classificamos manutenibilidade, testabilidade e complexidade operacional em suportabilidade (S); desempenho e escalabilidade em desempenho (P). A previsão de 100 mil usuários não informa simultaneidade nem comprova um gargalo. Precisamos medir carga e latência antes de distribuir o Core.

3. ASR — seleção obrigatória

Escolha no máximo 3 RNFs acima que realmente dirigem esta decisão. Para cada um, explique em uma frase como ele pode alterar estrutura, comunicação, dados, implantação ou operação.

Resposta da equipe

D1-A1 - Manutenibilidade (S): Localizar mudanças no módulo de ranking, restringindo acesso à persistência. Verificação: Domínio de pontuação sem imports de Spring/JPA; revisão de dependências.

D1-A2 - Escalabilidade (P): Preservar fronteira para extrair ranking apenas se a carga justificar outro deploy. Verificação: Meta proposta: p95 de leitura < 300 ms sob carga definida na AF; não validada para 100 mil usuários.

D1-A3 - Complexidade operacional (S): Manter um deploy e um banco para equipe de seis pessoas. Verificação: Contagem dos processos essenciais do Core: um.

4. RPC — restrições e premissas do desafio

Core atual em Java/Spring Boot; equipe de 6 desenvolvedores; banco relacional e legado da AC1.

Resposta da equipe

O cenário exige Java/Spring Boot, banco relacional, preservação do legado da AC1 e considera uma equipe de seis desenvolvedores. Evoluímos o MVP educacao-continuada-gamificada da equipe na pasta ac1, preservando promoção, recompensas e os testes originais. O commit de origem e a comparação antes/depois estão no registro arquitetural.

5. Decisão do Tech Lead

Quais RNFs tornam-se ASRs neste cenário? Há necessidade comprovada de escala/deploy independente? A complexidade distribuída é justificável?

Resposta da equipe

Escolhemos o monólito modular e mantivemos o ranking no mesmo processo. O critério de revisão é saturação comprovada do ranking, apesar de paginação, índices e otimização, ou necessidade frequente de deploy independente. A equipe de seis pessoas favorece reduzir componentes operacionais enquanto não há essa evidência.

6. Alternativas e trade-offs

Comparar: (A) Monólito em Camadas; (B) Monólito Modular; (C) decomposição seletiva em serviço(s).

Resposta da equipe

A - Camadas: operação simples, mas mudanças em uma capacidade atravessam pastas horizontais. B - Monólito modular: agrupa capacidade e explicita limites mantendo uma transação local; requer disciplina de dependências. C - Extração seletiva de ranking: escala e deploy próprios, mas adiciona contrato remoto, consistência eventual e operação. O volume total de usuários, isoladamente, não prova a necessidade de C.

7. Mini-lab / evidência

Reorganizar uma funcionalidade da AC1 em uma pequena estrutura modular e comparar dependências antes/depois.

Resposta da equipe

origem_ac1.json e comparacao_ac1.json registram a base real e as dependências. Movemos RecompensaService, RecompensaRepository, RecompensaEntity e RecompensaResponse para o módulo recompensa. O novo ranking usa dados reais de aluno, cursos e moedas, com porta, domínio puro e adapter JPA. Os testes originais de promoção e recompensas foram preservados.

Consultar as evidências deste laboratório

8. ADR + C4

Registrar ou atualizar um ADR resumido: Contexto → ASRs/RPCs → Alternativas → Decisão → Consequências/trade-offs. C4 Container. Se a decisão for apenas interna ao Core, indicar o impacto e preparar o Component para o Desafio 2.

Resposta da equipe

ADR-001 — Aceito para a PoC didática. Contexto: o cenário descrito neste desafio. A decisão considera os ASRs e RPCs registrados nos itens 3 e 4 e a comparação do item 6.

Decisão: Escolhemos o monólito modular e mantivemos o ranking no mesmo processo. O critério de revisão é saturação comprovada do ranking, apesar de paginação, índices e otimização, ou necessidade frequente de deploy independente. A equipe de seis pessoas favorece reduzir componentes operacionais enquanto não há essa evidência.

Consequências: Um único deploy continua sendo ponto comum de falha e escala. O adapter de ranking da AC1 ainda lê todos os alunos; a modularização não elimina esse custo nem comprova capacidade para 100 mil usuários. Paginação e projeção de leitura são próximos experimentos. O módulo de recompensas mantém relação com AlunoEntity e evento AlunoPromovidoEvent, dependências explícitas que ainda exigem disciplina.

C4: Container: navegador, Core e banco permanecem. Component: ranking passa a ter API, serviço, porta, regra de pontuação e adapter JPA.

ADR-001 no repositório

Diagramas C4 e identificação dos componentes implementados e propostos

Entrega do desafio — 1,0 ponto

Atualizar /docs/arquitetura/README.md e versionar a evidência. A decisão deve permitir rastrear: RF → RNF → ASR + RPC → alternativas/trade-offs → ADR → C4 → evidência.

DESAFIO 2 — ORGANIZAÇÃO INTERNA DO CORE

Proteger o domínio contra dependências técnicas

Cenário / problema

As regras de ranking, moedas, badges e recompensas mudam com frequência. A equipe precisa evitar que domínio, banco, framework e integrações externas fiquem fortemente acoplados.

1. RF — o que o sistema precisa fazer

RF01 Ranking; RF02 Conquistas e regras de gamificação.

Resposta da equipe

Isolamos o cálculo de ranking, preservando a regra cursos concluídos × 100 + moedas. As regras de conquistas e recompensas devem continuar pertencendo ao domínio, sem depender da tecnologia de persistência.

2. RNF — qualidades a analisar

Manutenibilidade, testabilidade, modificabilidade e facilidade de evolução.

Resposta da equipe

Manutenibilidade, modificabilidade, testabilidade e facilidade de evolução pertencem à suportabilidade (S). A prioridade é alterar regras sem reescrever a infraestrutura e testar o domínio sem iniciar banco ou Spring.

3. ASR — seleção obrigatória

Escolha no máximo 3 RNFs acima que realmente dirigem esta decisão. Para cada um, explique em uma frase como ele pode alterar estrutura, comunicação, dados, implantação ou operação.

Resposta da equipe

D2-A1 - Modificabilidade (S): Inverter a dependência da aplicação para uma porta de leitura. Verificação: RankingService depende de RankingPort, não do repository.

D2-A2 - Testabilidade (S): Executar regra e caso de uso sem inicializar Spring nem banco. Verificação: Pontuacao e fake testados com JUnit; mesmo resultado com JPA/H2.

4. RPC — restrições e premissas do desafio

Core continua Spring Boot; regras existentes devem ser reaproveitadas; não é permitido reescrever o produto inteiro apenas para aplicar um padrão.

Resposta da equipe

Spring Boot permanece; regras existentes devem ser reaproveitadas; não reescrever o produto inteiro. Preservamos as regras originais de promoção e recompensas. Adicionamos ranking com fórmula cursos concluídos × 100 + moedas e desempate por ID.

5. Decisão do Tech Lead

Qual dependência realmente precisa ser invertida? O custo estrutural é proporcional ao problema? DDD, Hexagonal, Clean e Onion são concorrentes ou podem ser combinados?

Resposta da equipe

Aplicamos Ports & Adapters somente ao ranking. RankingPort pertence à aplicação; RankingJpaAdapter implementa a porta; Pontuacao é Java puro. A aplicação usa @Service e a transação de leitura pertence ao adapter JPA: é uma adoção incremental de Ports & Adapters.

6. Alternativas e trade-offs

Comparar ao menos duas estruturas: arquitetura atual versus Ports & Adapters/Hexagonal ou estrutura equivalente; justificar eventual uso de DDD/Clean/Onion.

Resposta da equipe

A - Serviço diretamente ligado a AlunoRepository e entidade AlunoEntity: menos tipos, mas testes e mudanças de banco afetam a aplicação. B - Porta de leitura RankingPort e adapter JPA: a aplicação recebe dados simples, e o domínio calcula os pontos; adiciona mapeamento e uma interface. DDD ajuda a nomear capacidades e regras; Hexagonal define portas/adapters; Clean e Onion descrevem direção de dependências. Não são produtos obrigatórios nem padrões mutuamente exclusivos.

7. Mini-lab / evidência

Isolar uma regra de domínio e trocar pelo menos um adapter (ex.: persistência fake/mock versus JPA), mantendo testes do domínio.

Resposta da equipe

RankingTest valida pontuação e ordenação com porta fake. RankingIntegrationTest usa AlunoService real: conclui 12 cursos da AC1, promove o aluno e verifica 1203 pontos no adapter JPA. Pontuacao testa negativos e overflow. A regra não importa Spring ou JPA.

Consultar as evidências deste laboratório

8. ADR + C4

Registrar ou atualizar um ADR resumido: Contexto → ASRs/RPCs → Alternativas → Decisão → Consequências/trade-offs. C4 Component do Core, mostrando responsabilidades e fronteiras relevantes.

Resposta da equipe

ADR-002 — Aceito para a PoC didática. Contexto: o cenário descrito neste desafio. A decisão considera os ASRs e RPCs registrados nos itens 3 e 4 e a comparação do item 6.

Decisão: Aplicamos Ports & Adapters somente ao ranking. RankingPort pertence à aplicação; RankingJpaAdapter implementa a porta; Pontuacao é Java puro. A aplicação usa @Service e a transação de leitura pertence ao adapter JPA: é uma adoção incremental de Ports & Adapters.

Consequências: Trocar persistência exige implementar a porta, sem mudar a regra. Existem mais tipos e mapeamentos. A ordenação é feita em memória pelo resultado calculado no domínio. Paginação e projeção de leitura serão avaliadas após medir carga.

C4: Component do Core mostra controller → serviço → porta; adapter JPA implementa porta e acessa repository; serviço chama Pontuacao.

ADR-002 no repositório

Diagramas C4 e identificação dos componentes implementados e propostos

Entrega do desafio — 1,0 ponto

Atualizar /docs/arquitetura/README.md e versionar a evidência. A decisão deve permitir rastrear: RF → RNF → ASR + RPC → alternativas/trade-offs → ADR → C4 → evidência.

DESAFIO 3 — SERVIÇO PYTHON E IA

Definir a fronteira arquitetural de ML e RAG/LLM

Cenário / problema

A plataforma incorpora recomendação inteligente e assistente educacional. Os modelos serão desenvolvidos em Python, terão ciclo de atualização próprio, carga variável e poderão ficar indisponíveis sem impedir operações essenciais do Core.

1. RF — o que o sistema precisa fazer

RF03 Recomendação inteligente; RF04 Assistente educacional.

Resposta da equipe

RF03 deve fornecer recomendações de cursos e RF04 deve oferecer assistência educacional. No mini-lab, exercitamos a integração de recomendação com uma resposta determinística; modelo treinado e RAG completo ficam para a AF.

2. RNF — qualidades a analisar

Deploy independente, escalabilidade, tolerância a falhas, interoperabilidade, desempenho e manutenibilidade.

Resposta da equipe

Deploy independente e manutenibilidade são suportabilidade (S); tolerância a falhas é confiabilidade (R); desempenho e escalabilidade são desempenho (P). A interoperabilidade exige um contrato estável entre Java e Python.

3. ASR — seleção obrigatória

Escolha no máximo 3 RNFs acima que realmente dirigem esta decisão. Para cada um, explique em uma frase como ele pode alterar estrutura, comunicação, dados, implantação ou operação.

Resposta da equipe

D3-A1 - Deploy independente (S): Separar o processo Python do Core para publicar modelos sem recompilar Java. Verificação: Dockerfiles separados; endpoint FastAPI executado em processo distinto.

D3-A2 - Tolerância a falhas (R): Limitar espera da IA e responder com catálogo quando ela falhar. Verificação: HTTP 200 com origem python e, após queda, origem catalogo.

D3-A3 - Escalabilidade (P): Permitir ampliar réplicas da IA sem replicar todo o Core. Verificação: Fronteira de processo presente; teste de carga ainda necessário.

4. RPC — restrições e premissas do desafio

Core permanece Java/Spring Boot; ML/RAG utilizam Python; indisponibilidade da IA não pode bloquear o fluxo essencial.

Resposta da equipe

Core em Java/Spring Boot; ML e RAG em Python; a indisponibilidade da IA não pode bloquear operações essenciais. O mini-lab usa uma resposta determinística de recomendação para testar a fronteira; não contém modelo treinado nem uma implementação completa de RAG.

5. Decisão do Tech Lead

Python por si só justifica um serviço? ML e RAG precisam da mesma fronteira? Quais ASRs justificam autonomia e escala? Qual comportamento degradado é aceitável?

Resposta da equipe

Escolhemos um serviço Python independente, com endpoints e responsabilidades separáveis. ML e RAG começam no mesmo limite operacional; extrair RAG se consumo de memória, latência ou cadência de atualização divergirem. A linguagem Python sozinha não justifica um serviço: a decisão depende dos três ASRs. Python por si só não exige um serviço: a fronteira atende autonomia de deploy, escala e tolerância a falhas. O fallback aceitável é o catálogo básico; o assistente deve informar indisponibilidade.

6. Alternativas e trade-offs

Comparar: (A) IA embutida no Core; (B) módulo acoplado; (C) serviço Python independente; discutir se ML e RAG ficam juntos ou separados.

Resposta da equipe

A - IA embutida: menos rede, porém runtime e atualização de modelos prendem-se ao Core. B - Módulo acoplado/script local: interoperação possível, mas compartilha ciclo de implantação e recursos. C - Serviço Python: contrato remoto e falhas de rede, porém deploy e capacidade independentes. Separar ML e RAG já de início duplica operação sem medição; mantê-los juntos aceita disputa interna por recursos.

7. Mini-lab / evidência

Spring Boot consumindo FastAPI; interromper o serviço Python e observar/tratar a falha.

Resposta da equipe

ia-online.log e ia-offline.log registram o mesmo endpoint Spring respondendo 200: primeiro origem python, depois origem catalogo com o processo FastAPI interrompido. RecomendacaoTest exercita HTTP real em loopback: sucesso, erro 503 e serviço interrompido, verificando fallback e contador na AC1.

Consultar as evidências deste laboratório

8. ADR + C4

Registrar ou atualizar um ADR resumido: Contexto → ASRs/RPCs → Alternativas → Decisão → Consequências/trade-offs. C4 Container com a fronteira da IA e protocolos de comunicação.

Resposta da equipe

ADR-003 — Aceito para a PoC didática. Contexto: o cenário descrito neste desafio. A decisão considera os ASRs e RPCs registrados nos itens 3 e 4 e a comparação do item 6.

Decisão: Escolhemos um serviço Python independente, com endpoints e responsabilidades separáveis. ML e RAG começam no mesmo limite operacional; extrair RAG se consumo de memória, latência ou cadência de atualização divergirem. A linguagem Python sozinha não justifica um serviço: a decisão depende dos três ASRs.

Consequências: Recomendação indisponível devolve catálogo básico; assistente deve informar indisponibilidade, nunca inventar resposta. Cliente Java tem timeout de conexão de 300 ms e de requisição de 800 ms, sem retries no caminho interativo. Instrumentamos um contador para cada fallback. Circuit breaker e limite de concorrência ficam para a AF, após observar falhas sustentadas.

C4: Container adiciona serviço Python e HTTP/JSON Core → IA. Vetores/documentos do RAG são capacidade futura, não artefatos do mini-lab.

ADR-003 no repositório

Diagramas C4 e identificação dos componentes implementados e propostos

Entrega do desafio — 1,0 ponto

Atualizar /docs/arquitetura/README.md e versionar a evidência. A decisão deve permitir rastrear: RF → RNF → ASR + RPC → alternativas/trade-offs → ADR → C4 → evidência.

DESAFIO 4 — COMUNICAÇÃO E EVENTOS

Decidir onde sincronismo é necessário e onde eventos reduzem acoplamento

Cenário / problema

Ao concluir um curso, o sistema registra a conclusão, calcula recompensas, atualiza informações e pode disparar notificação/analytics. Uma falha de notificação não pode impedir a conclusão do curso.

1. RF — o que o sistema precisa fazer

RF02 Conquistas; RF05 Notificações; eventos associados à conclusão de atividades.

Resposta da equipe

A conclusão de curso deve registrar o progresso e calcular recompensas. RF05 prevê notificações por e-mail e SMS como efeitos posteriores: a falha de qualquer canal não deve desfazer a conclusão. No B1, validamos a estratégia de entrega; o teste usa um consumidor HTTP.

2. RNF — qualidades a analisar

Tolerância a falhas, disponibilidade, latência, consistência, desacoplamento e recuperabilidade.

Resposta da equipe

Disponibilidade, tolerância a falhas, consistência e recuperabilidade são confiabilidade (R); latência é desempenho (P); desacoplamento é suportabilidade (S). Aceitamos atraso na notificação, mas não a perda silenciosa da intenção de enviá-la.

3. ASR — seleção obrigatória

Escolha no máximo 3 RNFs acima que realmente dirigem esta decisão. Para cada um, explique em uma frase como ele pode alterar estrutura, comunicação, dados, implantação ou operação.

Resposta da equipe

D4-A1 - Disponibilidade (R): Retirar notificação do caminho síncrono da conclusão. Verificação: Conclusão permanece gravada com consumidor indisponível.

D4-A2 - Consistência (R): Gravar conclusão e intenção de notificar na mesma transação. Verificação: Outbox SQLite persiste junto com conclusão.

D4-A3 - Recuperabilidade (R): Reentregar pendências e deduplicar identificadores no consumidor. Verificação: Reabertura do banco recupera evento; uma única entrega lógica.

4. RPC — restrições e premissas do desafio

Operações essenciais do Core devem concluir mesmo com consumidor externo indisponível; adoção de broker exige custo operacional.

Resposta da equipe

Consumidor externo indisponível não pode impedir a conclusão; broker adiciona custo de operação. A microsolução usa SQLite para demonstrar atomicidade local e durabilidade, sem simular uma implantação completa em PostgreSQL.

5. Decisão do Tech Lead

Qual resposta precisa ser imediata? Qual pode ser eventual? Que ASR justificaria mensageria? RabbitMQ/Kafka resolvem um problema real do fluxo?

Resposta da equipe

Decidimos manter conclusão/recompensa no Core e notificação eventual via outbox. Não adicionar RabbitMQ/Kafka neste estágio. O worker deve confirmar sucesso depois do consumidor e repetir pendências. Event ID idempotente no consumidor garante um único efeito mesmo com entrega repetida. O retorno de conclusão e recompensa é imediato; e-mail, SMS e analytics são eventuais. RabbitMQ passa a fazer sentido com vários consumidores e roteamento; Kafka, se retenção e replay forem requisitos demonstrados.

6. Alternativas e trade-offs

Comparar REST síncrono com comunicação assíncrona; se houver broker, comparar RabbitMQ/Kafka pelo problema do case, não por popularidade.

Resposta da equipe

A - REST síncrono: resposta imediata e fluxo simples, mas a indisponibilidade externa propaga falha. B - Outbox transacional e worker com entrega HTTP: grava intenção junto ao estado, permite retry, adiciona agendamento e controle de duplicidade. C - Outbox + RabbitMQ: útil para filas, roteamento e vários workers. Kafka favorece retenção/replay e múltiplos fluxos de consumo, mas exige governança e operação desproporcionais à notificação atual.

7. Mini-lab / evidência

Implementar uma interação síncrona e uma assíncrona simples; simular consumidor indisponível e registrar o efeito.

Resposta da equipe

microsolucoes/eventos.py: falha síncrona é propagada; conclusão+outbox permanecem gravadas quando a entrega falha; reabrir banco e reentregar recupera a pendência; repetir o mesmo event ID produz um único registro recebido.

Consultar as evidências deste laboratório

8. ADR + C4

Registrar ou atualizar um ADR resumido: Contexto → ASRs/RPCs → Alternativas → Decisão → Consequências/trade-offs. C4 Container + sequência simplificada do fluxo crítico, se necessária para demonstrar a decisão.

Resposta da equipe

ADR-004 — Aceito para a PoC didática. Contexto: o cenário descrito neste desafio. A decisão considera os ASRs e RPCs registrados nos itens 3 e 4 e a comparação do item 6.

Decisão: Decidimos manter conclusão/recompensa no Core e notificação eventual via outbox. Não adicionar RabbitMQ/Kafka neste estágio. O worker deve confirmar sucesso depois do consumidor e repetir pendências. Event ID idempotente no consumidor garante um único efeito mesmo com entrega repetida.

Consequências: Sem exactly once: queda entre envio e marcação pode repetir a entrega. A AF deve persistir deduplicação no consumidor, definir backoff, limite de tentativas, fila de falhas e retenção. A notificação pode atrasar e o estado operacional de pendências precisa ser medido. O lab não altera ainda o fluxo de matrícula do Core.

C4: Container alvo inclui worker de outbox e consumidor de notificação. A sequência distingue commit local de entrega posterior. Estes componentes estão no lab de eventos, ainda separados da aplicação Spring.

ADR-004 no repositório

Diagramas C4 e identificação dos componentes implementados e propostos

Entrega do desafio — 1,0 ponto

Atualizar /docs/arquitetura/README.md e versionar a evidência. A decisão deve permitir rastrear: RF → RNF → ASR + RPC → alternativas/trade-offs → ADR → C4 → evidência.

DESAFIO 5 — IoT E MQTT

Integrar dispositivos com recursos limitados e conectividade intermitente

Cenário / problema

Laboratórios passam a produzir eventos de presença, início e conclusão de atividades por dispositivos com recursos limitados e conexão instável. Esses eventos devem entrar na plataforma sem transformar o Core em um acoplamento direto com cada dispositivo.

1. RF — o que o sistema precisa fazer

RF06 Receber eventos IoT e encaminhá-los ao ecossistema da plataforma.

Resposta da equipe

Recebemos eventos de dispositivos, identificamos a instituição e transformamos o payload em um evento da aplicação. O mini-lab transforma presença em PresencaRegistrada, preparando a futura ligação com o Core.

2. RNF — qualidades a analisar

Interoperabilidade, eficiência de comunicação, disponibilidade, tolerância à conectividade intermitente, escalabilidade e segurança de integração.

Resposta da equipe

Eficiência e escalabilidade são desempenho (P); disponibilidade e tolerância à desconexão são confiabilidade (R); interoperabilidade é suportabilidade (S); segurança integra funcionalidade (F). O protocolo deve atender dispositivos limitados sem expor diretamente o Core.

3. ASR — seleção obrigatória

Escolha no máximo 3 RNFs acima que realmente dirigem esta decisão. Para cada um, explique em uma frase como ele pode alterar estrutura, comunicação, dados, implantação ou operação.

Resposta da equipe

D5-A1 - Eficiência de comunicação (P): Usar publicação MQTT com mensagens pequenas em vez de polling HTTP. Verificação: Publicador e consumidor QoS 1 executados em broker local.

D5-A2 - Conectividade intermitente (R): Exigir identificador estável, buffer e reenvio no dispositivo. Verificação: Duplicata testada; buffer persistente e sessão durável ainda são trabalho da AF.

D5-A3 - Segurança de integração (F+): Isolar tópico por instituição e validar identidade e payload no gateway. Verificação: Tópico institucional demonstrado; TLS/ACL ainda não exercitados no lab local.

4. RPC — restrições e premissas do desafio

Dispositivos podem ter baixo poder computacional; conectividade pode ser intermitente; protocolo e broker adicionam componentes operacionais.

Resposta da equipe

Dispositivos podem ter CPU/memória limitadas e conexão instável. Broker e protocolo adicionam operação. O mini-lab usa broker real em loopback e modo anônimo apenas em ambiente local isolado.

5. Decisão do Tech Lead

REST é suficiente? O que MQTT resolve especificamente? Após MQTT, existe motivo arquitetural para outro broker? Como impedir duplicidade ou perda relevante de eventos?

Resposta da equipe

Escolhemos MQTT QoS 1 com gateway que transforma o contrato do dispositivo. Tópico por instituição, event_id estável e consumidor idempotente. Não adicionar outro broker. Em produção, usar TLS, credenciais por dispositivo, ACL por tópico, validação de esquema e deduplicação persistente antes de aplicar o evento no Core. REST seria suficiente para poucos dispositivos com conexão estável. MQTT atende publicação desacoplada e recursos limitados. QoS 1 admite repetição; evitar perda em desconexões exige também buffer persistente e sessões duráveis, ainda não validados.

6. Alternativas e trade-offs

Comparar: (A) REST direto; (B) MQTT + consumidor; (C) MQTT + integração posterior com mensageria interna, somente se houver ASR que justifique.

Resposta da equipe

A - REST direto: infraestrutura conhecida, mas dispositivos precisam implementar retry e conexão por requisição; Core fica exposto ao ritmo dos dispositivos. B - MQTT + gateway: publicação leve, QoS e desacoplamento; demanda broker, autenticação e gestão de sessão. C - MQTT + outro broker interno: justificável quando vários consumidores precisam replay/roteamento independente; hoje duplica a operação sem requisito demonstrado.

7. Mini-lab / evidência

Simulador publica evento em broker MQTT; consumidor recebe e transforma o evento para uso da aplicação.

Resposta da equipe

microsolucoes/mqtt_lab.py e mqtt.log: publicador envia duas cópias com QoS 1 ao broker; gateway recebe duas mensagens e gera uma PresencaRegistrada. A transformação ocorre antes da futura integração com o Core.

Consultar as evidências deste laboratório

8. ADR + C4

Registrar ou atualizar um ADR resumido: Contexto → ASRs/RPCs → Alternativas → Decisão → Consequências/trade-offs. C4 Container mostrando dispositivos, broker, consumidor/gateway e Core; fluxo do evento quando útil.

Resposta da equipe

ADR-005 — Aceito para a PoC didática. Contexto: o cenário descrito neste desafio. A decisão considera os ASRs e RPCs registrados nos itens 3 e 4 e a comparação do item 6.

Decisão: Escolhemos MQTT QoS 1 com gateway que transforma o contrato do dispositivo. Tópico por instituição, event_id estável e consumidor idempotente. Não adicionar outro broker. Em produção, usar TLS, credenciais por dispositivo, ACL por tópico, validação de esquema e deduplicação persistente antes de aplicar o evento no Core.

Consequências: QoS 1 admite duplicatas e não substitui buffer persistente do dispositivo. O mini-lab demonstra deduplicação em memória e transformação, mas não comprova recuperação depois de reinício nem segurança multi-instituição. Esses itens são critérios de aceitação da AF, e não capacidades alegadas como já implementadas.

C4: Container alvo: dispositivo → broker MQTT → gateway → Core. MQTT/TCP no lab; MQTT/TLS e HTTPS na implantação proposta.

ADR-005 no repositório

Diagramas C4 e identificação dos componentes implementados e propostos

Entrega do desafio — 1,0 ponto

Atualizar /docs/arquitetura/README.md e versionar a evidência. A decisão deve permitir rastrear: RF → RNF → ASR + RPC → alternativas/trade-offs → ADR → C4 → evidência.

DESAFIO 6 — CI/CD E OBSERVABILIDADE

Provar que a arquitetura pode ser construída, implantada e observada

Cenário / problema

Após as decisões anteriores, a solução pode conter Core Java, serviço(s) Python, broker e componentes de integração. A arquitetura só é sustentável se a equipe conseguir testar, implantar e observar suas partes com segurança.

1. RF — o que o sistema precisa fazer

Todos os RFs impactados pelos componentes construídos; operação e evolução da solução.

Resposta da equipe

Precisamos construir, testar, implantar e acompanhar os componentes que suportam os RFs anteriores. A operação deve identificar falhas e degradação da recomendação, mesmo quando o endpoint continua respondendo.

2. RNF — qualidades a analisar

Observabilidade, deploy independente, confiabilidade, testabilidade, recuperabilidade e desempenho.

Resposta da equipe

Observabilidade e testabilidade são suportabilidade (S); confiabilidade e recuperação de falhas são confiabilidade (R); desempenho é P. Deploy independente exige artefatos e configuração separados, embora a orquestração inicial da esteira possa ser única.

3. ASR — seleção obrigatória

Escolha no máximo 3 RNFs acima que realmente dirigem esta decisão. Para cada um, explique em uma frase como ele pode alterar estrutura, comunicação, dados, implantação ou operação.

Resposta da equipe

D6-A1 - Observabilidade (S): Instrumentar falhas recuperadas, além do status HTTP. Verificação: ia_fallback_total aumenta mesmo quando a API responde 200.

D6-A2 - Confiabilidade de entrega (R): Bloquear implantação quando testes ou build falharem. Verificação: Jenkinsfile sequencial sem ignorar falhas; Jenkins executou os testes, builds e deploy do laboratório; 64 testes aprovados, nenhum ignorado.

D6-A3 - Recuperabilidade operacional (R): Versionar imagens/configuração e preparar retorno à versão anterior. Verificação: JAR e configurações versionáveis; rollback em ambiente Jenkins ainda pendente.

4. RPC — restrições e premissas do desafio

Jenkins é a base da esteira; Prometheus/Grafana são as ferramentas do laboratório; artefatos e configurações devem ser versionados.

Resposta da equipe

Jenkins é a base da esteira e Prometheus/Grafana são ferramentas do laboratório. Artefatos e configuração versionados. Esta entrega inclui a evolução pedida no B1; a atividade A2 de Jenkins não foi refeita nem sobrescrita.

5. Decisão do Tech Lead

Serviços independentes exigem pipelines independentes? O que deve bloquear deploy? Quais métricas validam ASRs anteriores? Como distinguir métrica técnica de evidência arquitetural?

Resposta da equipe

Executamos um Jenkinsfile com teste/build antes de Docker e smoke test após implantação. Falhas Maven bloqueiam deploy; falha de saúde falha a execução. Prometheus coleta Actuator e Grafana consulta Prometheus. O contador ia_fallback_total evidencia degradação invisível ao HTTP 200. Serviços independentes não obrigam servidores Jenkins separados. Etapas e artefatos separados permitem evoluir para pipelines próprios. Testes, build e smoke test devem bloquear a promoção quando falharem. Aumento de ia_fallback_total após interromper Python demonstra o ASR de tolerância a falhas; HTTP 200 isolado não mostra essa degradação. Latência p95, taxa de erros e throughput precisam de carga definida para sustentar uma decisão de escala.

6. Alternativas e trade-offs

Comparar pipeline único versus pipelines por componente; selecionar métricas que comprovem decisões (latência, erros, disponibilidade, throughput etc.).

Resposta da equipe

A - Pipeline único: integração simples e coordenação central, mas mudanças na IA podem aguardar etapas do Core. B - Pipelines por componente: deploy independente e menor escopo de falha, com custo de contratos e promoção conjunta. Com um único repositório e equipe pequena, o pipeline inicial único tem etapas separadas; extrair pipelines quando a frequência independente for comprovada.

7. Mini-lab / evidência

Evoluir Jenkinsfile e instrumentar ao menos uma métrica; provocar uma condição observável e registrar evidência no Grafana/Prometheus.

Resposta da equipe

Execução https://github.com/Gustavo-Champam/Exercicio-B1-Arquitetura-de-Software-AC2/actions/runs/41537017845, resultado Jenkins SUCCESS. evidencias/jenkins/provenance.json vincula o resultado ao commit; junit.json registra 64 testes aprovados e nenhum ignorado. Os relatórios JaCoCo dos dois projetos registram 100% de linhas e ramos. validacao.json registra ranking real de 1203 pontos, IA online e fallback após parar o container, além dos contadores Prometheus. As capturas b1-grafana.png e a3-grafana.png mostram os painéis atuais. up comprova coleta, não disponibilidade funcional completa. Histogramas permitem medir latência; não alegamos teste de 100 mil usuários.

Consultar as evidências deste laboratório

8. ADR + C4

Registrar ou atualizar um ADR resumido: Contexto → ASRs/RPCs → Alternativas → Decisão → Consequências/trade-offs. C4 Deployment (ou Container com elementos de operação) + atualização final do C4 arquitetural.

Resposta da equipe

ADR-006 — Aceito para a PoC didática. Contexto: o cenário descrito neste desafio. A decisão considera os ASRs e RPCs registrados nos itens 3 e 4 e a comparação do item 6.

Decisão: Executamos um Jenkinsfile com teste/build antes de Docker e smoke test após implantação. Falhas Maven bloqueiam deploy; falha de saúde falha a execução. Prometheus coleta Actuator e Grafana consulta Prometheus. O contador ia_fallback_total evidencia degradação invisível ao HTTP 200.

Consequências: Jenkinsfile usa agente Linux com JDK 17 para o projeto, Python e Docker. O workflow inicia Jenkins com Java 21 e executa o Jenkinsfile obtido do Git. A validação inclui PostgreSQL real, build, cobertura, containers, APIs e métricas. O resultado efetivo e a proveniência devem ser consultados no artefato da execução. Não há rollback automático implementado; a AF deve usar tags imutáveis e validar retorno à versão anterior.

C4: Deployment representa Jenkins no runner temporário, containers da AC1, FastAPI, Prometheus/Grafana, PostgreSQL de teste e aplicação A3. Eventos e MQTT continuam microsoluções que comprovam as hipóteses dos desafios 4 e 5.

ADR-006 no repositório

Diagramas C4 e identificação dos componentes implementados e propostos

Entrega do desafio — 1,0 ponto

Atualizar /docs/arquitetura/README.md e versionar a evidência. A decisão deve permitir rastrear: RF → RNF → ASR + RPC → alternativas/trade-offs → ADR → C4 → evidência.

Síntese da equipe para a Avaliação Final

Levaremos a AC1 evoluída, o módulo de recompensas, o ranking com porta e adapter JPA, a fronteira Python com timeout/fallback e a esteira Jenkins. Integraremos a outbox ao fluxo de conclusão e o gateway MQTT ao Core; persistiremos deduplicação, configuraremos TLS/ACL e testaremos desconexão. Evoluiremos a resposta determinística para recomendação com modelo e RAG avaliado. Mediremos p95 e throughput com carga definida, isolaremos dados por instituição e validaremos rollback. Não adotaremos Kafka nem extrairemos ranking sem evidência de necessidade.

Registro cumulativo das seis decisões

Matriz cumulativa de ASRs